ALLOWING HTTP METHODS

When it comes to the HTTP method, there are only a handful of acceptable values, the most common of which are `HEAD`, `OPTIONS`, `GET`, `POST`, `PUT`, `PATCH`, and `DELETE`. If you'd like to allow all types of cross-origin requests, you can set the Access-Control-Allow-Methods header to all of these values, as follows:

```
var corsOptions = {
  allowMethods: ['HEAD', 'OPTIONS', 'GET', 'POST',
                 'PUT', 'PATCH', 'DELETE']
};
```

A better approach is to consider which types of requests your server allows, and only accept those types of requests. If your server only allows `GET` and `DELETE` requests, you should only set those values on the preflight:

```
var corsOptions = {
  allowMethods: [GET', DELETE']
};
```

Accurate preflight headers protect your server from unexpected requests, and serve as useful documentation for your clients. If your server doesn't allow `PUT` requests, there is no reason to accept them in the first place, only to have them fail on the actual request. From the client's perspective, if they try to make a `PUT` request and it fails on the preflight, the developer can look at the error message and clearly understand why the request is failing, as shown in figure 6.14. If the `PUT` request is accepted by the preflight request but then fails on the actual request, the developer may be misled into thinking that the `PUT` request is failing for some other reason.

Unlike HTTP methods, which have a small set of acceptable values, clients can send any HTTP header. Let's look at how to configure custom request headers.

ALLOWING HTTP HEADERS

It can be a bit tougher to know which HTTP headers to allow on requests because there are so many different request headers. If you have control over both the server and the client, you can control exactly which headers are included on requests. Because you control the server code, it's easy to add a new header to the `allow-Headers` whitelist.

If you're building an API for external clients, it can be harder to know which headers to allow. JavaScript libraries and frameworks may add their own custom headers to requests. Table 6.4 lists a few common HTTP request headers.

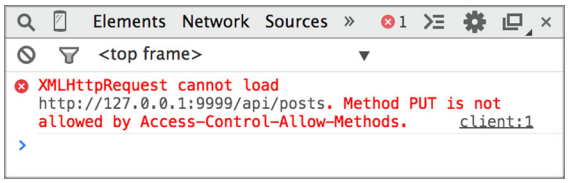


Figure 6.14 Example of a failing `PUT` request. The error message clearly indicates why the request is failing.